Department of Computer and Software Engineering
SE: Machine Learning

Course Instructor: Dr. Ahmed Raza	**Date:**
Day:	**Semester:**

Lab 5: Decision Tree Split using Gini and Information Gain

Lab Title	CLO	BT	PLO	Weightage
Decision Tree Split using Gini and Information Gain				

Name	Roll Number	Report /10	Viva /5	Total /15
Muhammad Abdul Qadir	BSSE23105			

Checked on: _____

Signature: _____

1 Learning Outcomes

After completing this lab, the student will be able to:

- Compute Gini impurity for binary classification.
- Calculate Information Gain from candidate splits.
- Implement the split selection procedure used by decision trees.
- Observe bias toward high-cardinality features.

2 Dataset

We will use the Iris dataset from sklearn. To simplify the problem we restrict the dataset to two classes.

Properties:

- Samples: 100
- Features: 4
- Classes: Binary

Dataset loading:

```
from sklearn.datasets import load_iris
```

3 Gini Impurity

For binary classification:

$$Gini = 1 - (p_0^2 + p_1^2)$$

A perfectly pure node has Gini impurity equal to 0.

4 Information Gain

The reduction in impurity after a split is:

$$Gain = Gini(parent) - \frac{n_L}{n} Gini(L) + \frac{n_R}{n} Gini(R)$$

The split with the highest information gain is selected.

5 Part A: Implement Gini Impurity (3 Marks)

Implement the function:

gini(y)

The function should:

- compute class probabilities
- apply the Gini formula
- return impurity

Verify using:

y = [0,0,0,1]

Compute impurity manually and compare.

Code:

```
def gini(y):
    """
    TODO

    Compute Gini impurity.

    Parameters
    ----------
    y : array-like

    Returns
    -------
    float
    """

    total_samples = len(y)

    if total_samples == 0:
        return 0.0

    class_counts = np.bincount(y)

    # Calculating the probability of each class
    class_prob=class_counts/total_samples
    # formula
    gini_impurity=1-np.sum(class_prob**2)

    return float(gini_impurity)
```

Verification using y=[0,0,0,1]

```
# Part A verification
    y_check = np.array([0, 0, 0, 1])

    manual = 1 - (0.75**2 + 0.25**2)
    from_function = gini(y_check)

    print("\n\nPart A: Gini Verification using y= [0, 0, 0, 1]")
    print("Manual: ", manual)
    print("Function: ", from_function)
    print("Are they both equal?: ", manual == from_function)
```

Comparison Output:

```
Part A: Gini Verification using y= [0, 0, 0, 1]
Manual:  0.375
Function:  0.375
Are they both equal?:  True
```

6 Part B: Implement Information Gain (4 Marks)

Implement:

information_gain(parent, left, right)

Steps:

- compute parent impurity
- compute child impurities
- compute weighted average
- subtract from parent impurity

```python
def information_gain(parent, left, right):
    """
    TODO

    Compute Information Gain.

    Parameters
    ----------
    parent : labels before split
    left : labels in left child
    right : labels in right child

    Returns
    -------
    float
    """

    total_samples=len(parent)
    no_left=len(left)
    no_right=len(right)

    left_weight=no_left/total_samples
    right_weight=no_right/total_samples

    weighted_child_impurity=(left_weight*gini(left) + right_weight*gini(right))

    # Gain is basically how much purer the children are as copared to the parent
    gain = gini(parent) - weighted_child_impurity
    return gain
```

7 Part C: Implement Tree Split Logic (4 Marks)

Implement the function:

find_best_split(X, y)

The function should:

- iterate over features
- compute candidate thresholds
- evaluate information gain
- return the best feature and threshold

Use NumPy operations where possible.

```python
def find_best_split(X, y):
    """
    TODO

    Implement decision tree split search.

    Steps
    -----
    1. Loop over features
    2. Generate candidate thresholds
    3. Split dataset
    4. Compute information gain
    5. Track best split

    Returns
    -------
    best_feature
    best_threshold
    best_gain
    """

    num_features=X.shape[1]
    best_feature=None
    best_threshold=None
    best_gain=-1.0

    for feature_index in range(num_features):
        feature_values = X[:, feature_index]
        thresholds = candidate_thresholds(feature_values)

        for threshold in thresholds:
            left_X, right_X, left_Y, right_Y = split_dataset(X, y, feature_index, threshold)

            if len(right_Y) == 0 or len(left_Y) == 0:
                continue

            gain = information_gain(y, left_Y, right_Y)
            if gain > best_gain:

                best_threshold=threshold
                best_gain=gain
                best_feature=feature_index

    return best_feature, best_threshold, best_gain
```

8 Part D: Cardinality Bias Experiment (4 Marks)

A new feature will be added:

index_feature = sample_index / N

This feature contains many unique values.
Run the split algorithm again and observe:

- which feature is selected
- whether the high-cardinality feature is preferred

Explain why this occurs.

```python
def add_high_cardinality_feature(X):
    """
    Add deterministic high-cardinality feature.

    Each sample receives a unique value derived
    from its index.

    This allows us to observe the bias of
    decision trees toward features with many
    unique values.
    """

    N = len(X)

    index_feature = (np.arange(N) / N).reshape(-1,1)

    return np.hstack([X, index_feature])
```

Output:

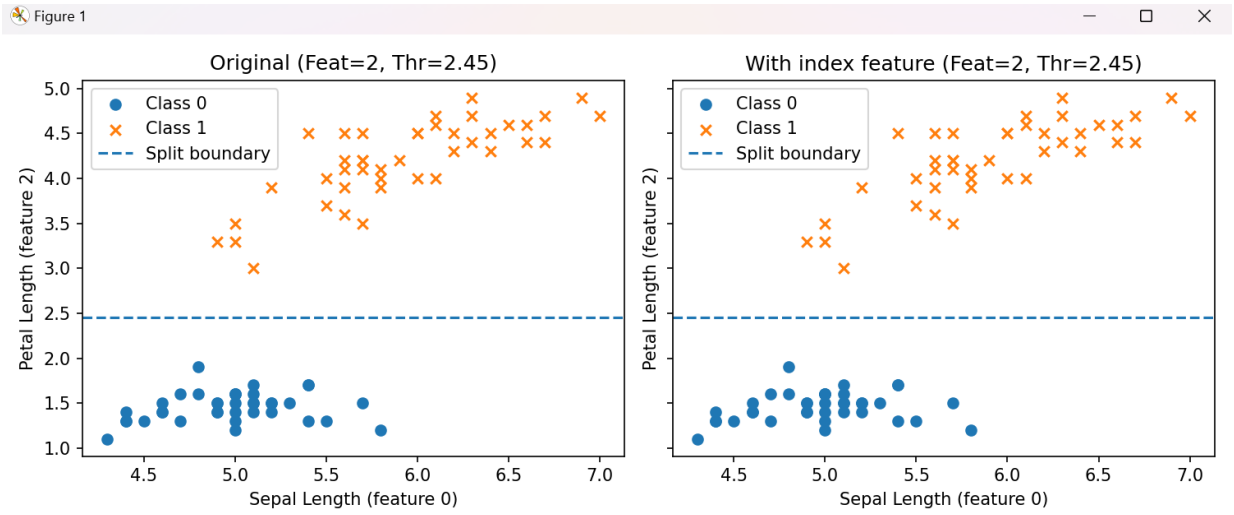
```
(MLlabtestenv) D:\ML-Labs\Lab05>python lab5_starter.py
Original Dataset
Best Feature: 2
Threshold: 2.45
Information Gain: 0.49875

With High Cardinality Feature
Best Feature: 2
Threshold: 2.45
Information Gain: 0.49875
```

Reason:

- After adding the high-cardinality index feature, the best split remained Feature 2 (petal length) and the index feature was not selected (Not preferred) because Feature 2 already provides the maximum impurity reduction, while The index feature is basically just the **row position**, and it doesn't have a real relationship with the class labels here, so it doesn't beat Feature 2.

Plots comparison



9 Expected Output

Your program should print:

- Best split feature
- Threshold
- Information gain
- Result of cardinality bias experiment

Assessment Rubrics

Method: Lab reports and instructor observation during lab sessions

Outcome Assessed:

1. Ability to conduct experiments, as well as to analyze and interpret data (P).
2. Ability to use the techniques, skills, and modern engineering tools necessary for engineering practice (P).

Performance	Exceeds expectation (4-5)	Meets expectation (3-2)	Does not meet expectation (1)	Marks
1. Realization of Experiment [a, b]	Selects relevant equipment to the experiment, develops setup diagrams of equipment connections or wiring.	Needs guidance to select relevant equipment and to develop equipment connection or wiring diagrams.	Incapable of selecting relevant equipment to conduct the experiment.	
2. Conducting Experiment [a, b]	Does proper calibration of equipment, carefully examines equipment moving parts, and ensures smooth operation.	Calibrates equipment, examines equipment moving parts, and operates the equipment with minor error.	Unable to calibrate appropriate equipment, and equipment operation is substantially wrong.	
3. Laboratory Safety Rules [a]	Respectfully and carefully observes safety rules and procedures.	Observes safety rules and procedures with minor deviation.	Disregards safety rules and procedures.	
6. Data Collection [a]	Plans data collection to achieve experimental objectives, and conducts an orderly and complete data collection.	Plans data collection to achieve experimental objectives, and collects complete data with minor error.	Does not know how to plan data collection; data collected is incomplete and contain errors.	
7. Data Analysis [a]	Accurately conducts simple computations and statistical analysis; correlates results to known theoretical values; accounts for measurement errors.	Conducts simple computations with minor error; reasonably correlates results to known theoretical values; attempts to account for errors.	Unable to conduct simple statistical analysis; no attempt to correlate or explain errors.	
8. Computer Use [a]	Uses computer to collect and analyze data effectively.	Uses computer to collect and analyze data with minor error.	Does not know how to use computer to collect and analyze data.	
			Total	

Faculty:

Name: _____

Signature: _____

Date: _____